Tema 5
433C
Grecu Victor

Notă: La crearea fișierelor si testarea lor acestea au fost create într-un singur folder, ulterior fiind încărcate pe Github în foldere separate.

https://github.com/IonGigi970/Tema5_CALP

- X = 6 (nr. litere din prenume), Y = 5 (nr. litere din nume)

1. Se consideră fișierul employee.py. atașat. Creați o clasă care va moșteni clasa Employee – clasa Manager.

- Numărul de obiecte create din clasa Manager va fi păstrat în variabila de clasă mgr_count
- Constructorul clasei va avea argumentele: name, salary, tasks și department
- La departament veți prefixa numele echipei voastre de la disciplină
- (X%4==2) În clasa Manager, modificați metoda 'display_employee' a.î. să afișeze doar departamentul.
- Creați un obiect (Y/3=1) al clasei Manager. Apelați metoda 'display_employee' pentru toate obiectele din clasa Manager și pentru obiectele din clasa Employee.
- Afișați valoarea atributului emp_count pentru o instanță a clasei Employee și pentru una a clasei Manager.

Prima dată am creat un folder .venv în folderul proiectului cu comanda python -m venv .venv scrisă în PowerShell, apoi am pornit mediul virtual cu .\.venv\Scripts\Activate.ps1. După aceea, am instalat pytest folosind comanda pip install -U pytest.

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\grecu\Desktop\Tema 5> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\grecu\Desktop\Tema 5> pip install -U pytest
Collecting pytest
  Using cached pytest-9.0.3-py3-none-any.whl.metadata (7.6 kB)
Requirement already satisfied: colorama>=0.4 in .\.venv\Lib\site-packages (from pytest) (0.4.6)
Requirement already satisfied: iniconfig>=1.0.1 in .\.venv\Lib\site-packages (from pytest) (2.3.0)
Requirement already satisfied: packaging>=22 in .\.venv\Lib\site-packages (from pytest) (26.2)
Requirement already satisfied: pluggy<2,>=1.5 in .\.venv\Lib\site-packages (from pytest) (1.6.0)
Requirement already satisfied: pygments>=2.7.2 in .\.venv\Lib\site-packages (from pytest) (2.20.0)
Using cached pytest-9.0.3-py3-none-any.whl (375 kB)
Installing collected packages: pytest
Successfully installed pytest-9.0.3

[notice] A new release of pip is available: 26.0.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\grecu\Desktop\Tema 5>
```

În fișierul manager.py este codul aferent cerinței. Am importat clasa Employee pentru ca manager să poată să o moștenească. Apoi am definit constructorul clasei în care am apelat, folosind funcția super(), constructorul din clasa Employees pentru atributele name și salary, restul fiind atribuite pe moment. Am modificat și funcția 'display_employee' care acum afișează doar departamentul, nu și numele + salariul. Pentru rulare, scriem python manager.py. Nu are importanță dacă suntem în venv sau nu, acolo instalăm doar modulele pe care le folosim în codul nostru ca să nu apară probleme/conflicte în cazul în care le instalăm la nivelul sistemului de operare. Output-ul poate fi observat în poza de mai jos :

```
PS C:\Users\grecu\Desktop\Tema 5> python .\manager.py
Department :  433C_IT
Name:  Mihai , Salary:  3000
Employee count accessed from Employee class: 3
Employee count accessed from Manager class: 3
Manager count: 1
PS C:\Users\grecu\Desktop\Tema 5>
```

De asemenea, au fost adăugate și funcțiile de testare de mai jos :

```
# Functii de testare
def is_tech_department(self):
    tech_dept = ['IT', 'Web Dev', 'Cybersecurity', 'Software Dev', 'Data Analytics']
    return any(dep in self.department for dep in tech_dept)
```

```
    def is_paid_more_than(self, other_employee):
        if not isinstance(other_employee, Employee):
            return False
        return self.salary > other_employee.salary

    def get_annual_salary(self):
        return self.salary * 12
```

Cu ele vom demonstra cum funcționează pytest pe care l-am instalat la început. Într-un fișier intitulat test_functions.py (codul se regăsește mai jos) am creat câte o funcție de test pentru metodele create anterior pentru clasa manager. A trebuit să import pytest, clasa Employee și Manager (prin import-urile de la începutul fișierului). Apoi am definit funcțiile de testare pentru a verifica dacă datele pe care le introducem corespund funcțiilor definite anterior. În fiecare din cele 3 funcții am creat câte un obiect al clasei manager sau employee (după caz), apoi am presupus că apelul funcției aferente din clasa Manager este adevărat folosind assert. În caz negativ, rezultatul va fi afișat în consolă cu roșu. La rularea cu comanda pytest, se poate observa rezultatul în poza de mai jos (trebuie să rulăm comanda pytest din venv, altfel primim eroare). Punctul ne spune că testul a funcționat și totul e corect, iar litera F ne spune care test a eșuat (a dat fail). Observăm că ultimul test nu a putut rula deoarece presupunerea noastră că salariul este 2000 (când în realitate era 6000) este falsă.

```
import pytest
from employee import Employee
from manager import Manager

def test_dept():
    man1 = Manager('George', 5000, 'Plot graphs', 'Data Analytics')
    assert man1.is_tech_department()
```

```
def test_paid_more():
    man2 = Manager('Aurel', 10000, 'Coding', 'Cybersecurity')
    emp1 = Employee('Maria', 2000)
    assert man2.salary > emp1.salary
def test_annual_salary():
    man3 = Manager('John', 500, 'Cleaning windows', 'Sanitation')
    assert man3.get_annual_salary() == 2000
```

```
Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS C:\Users\grecu\Desktop\Tema 5> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\grecu\Desktop\Tema 5> pytest
============================================= test session starts ==============================================
platform win32 -- Python 3.13.13, pytest-9.0.3, pluggy-1.6.0
rootdir: C:\Users\grecu\Desktop\Tema 5
collected 3 items

test_functions.py ..F                                                                                    [100%]

================================================= FAILURES =================================================
_____________________________________________ test_annual_salary _____________________________________________

    def test_annual_salary():
        man3 = Manager('John', 500, 'Cleaning windows', 'Sanitation')
>       assert man3.get_annual_salary() == 2000
E       assert 6000 == 2000
E        +  where 6000 = get_annual_salary()
E        +    where get_annual_salary = <manager.Manager object at 0x0000019BEC6CEB10>.get_annual_salary

test_functions.py:17: AssertionError
========================================== short test summary info ==========================================
FAILED test_functions.py::test_annual_salary - assert 6000 == 2000
======================================== 1 failed, 2 passed in 0.18s ========================================
(.venv) PS C:\Users\grecu\Desktop\Tema 5>
```

2. Se consideră fișierul data.csv atașat.

Să se importe valorile din fișier și să se ploteze (și afișeze grafic)

* toate valorile (pentru toate coloanele din csv: Durata, Puls, MaxPuls, Calorii)

* primele X=6 valori (pentru toate coloanele din csv: Durata, Puls, MaxPuls, Calorii)

* ultimele Y=5 valori (doar pentru coloanele Durata si Puls).

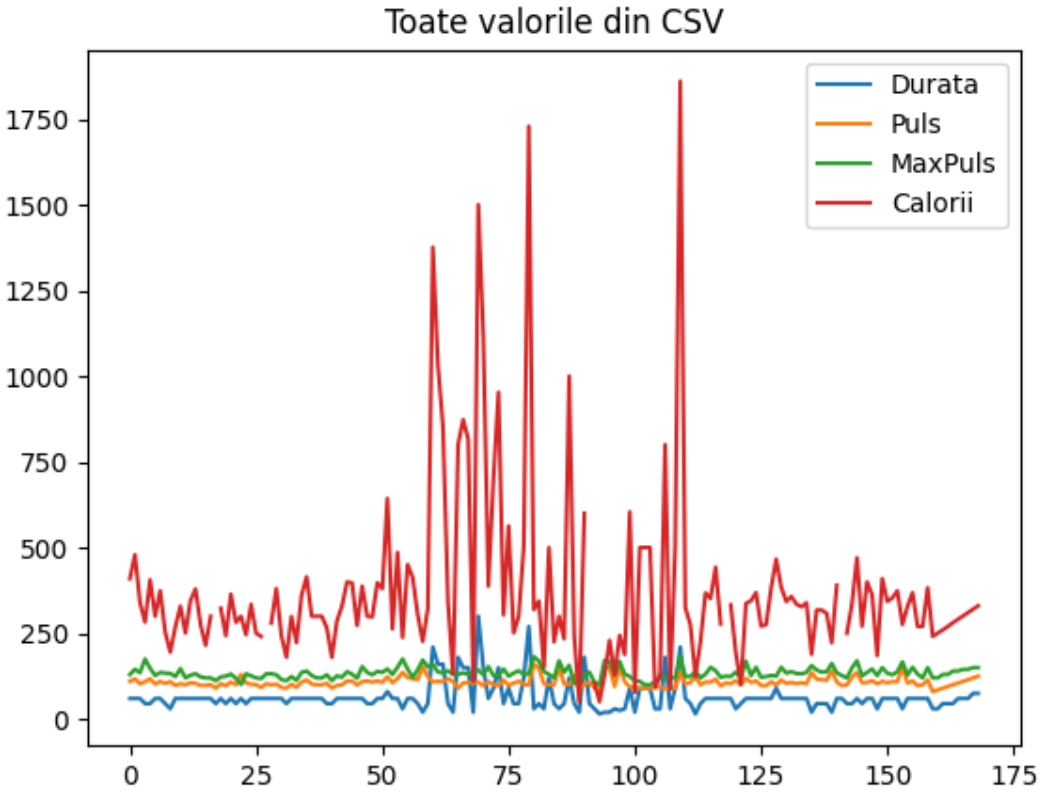
Hint: folosiți pandas pentru interacțiunea cu fișierul csv și selecția de date pentru diagrame, apoi matplotlib.pyplot pentru afișare.

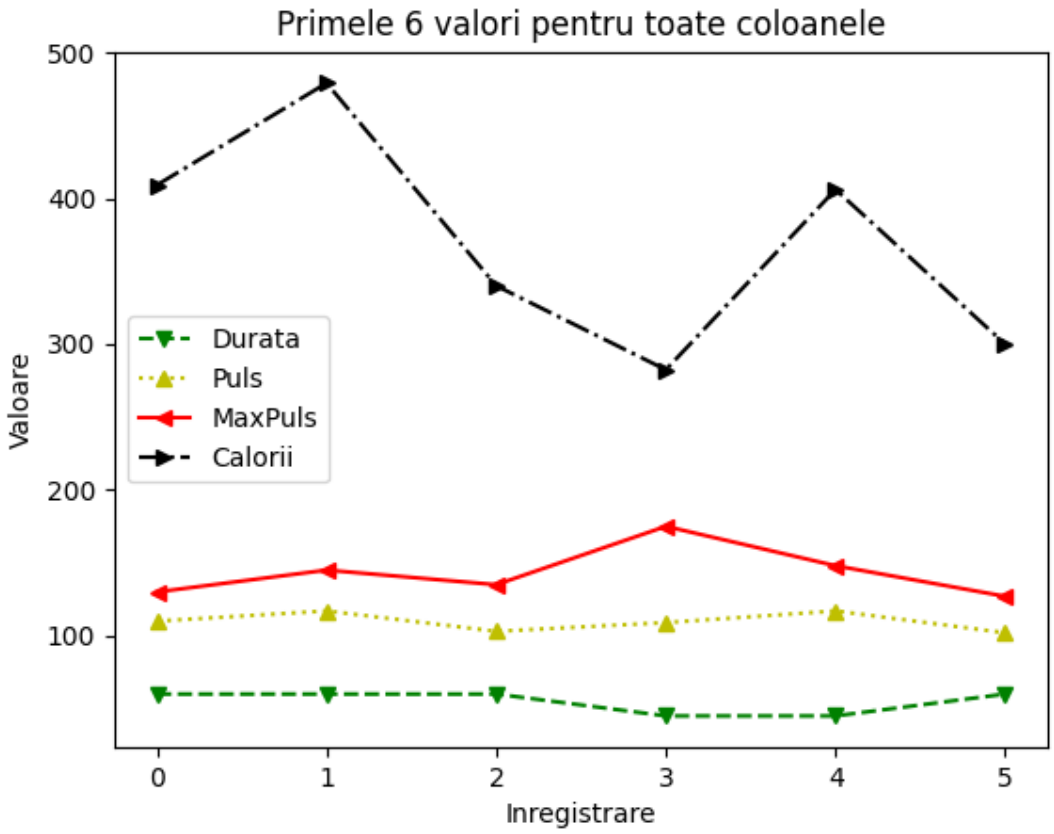
În venv-ul creat la început am rulat comanda pip install -U matplotlib pandas pentru a instala cele 2 librării.

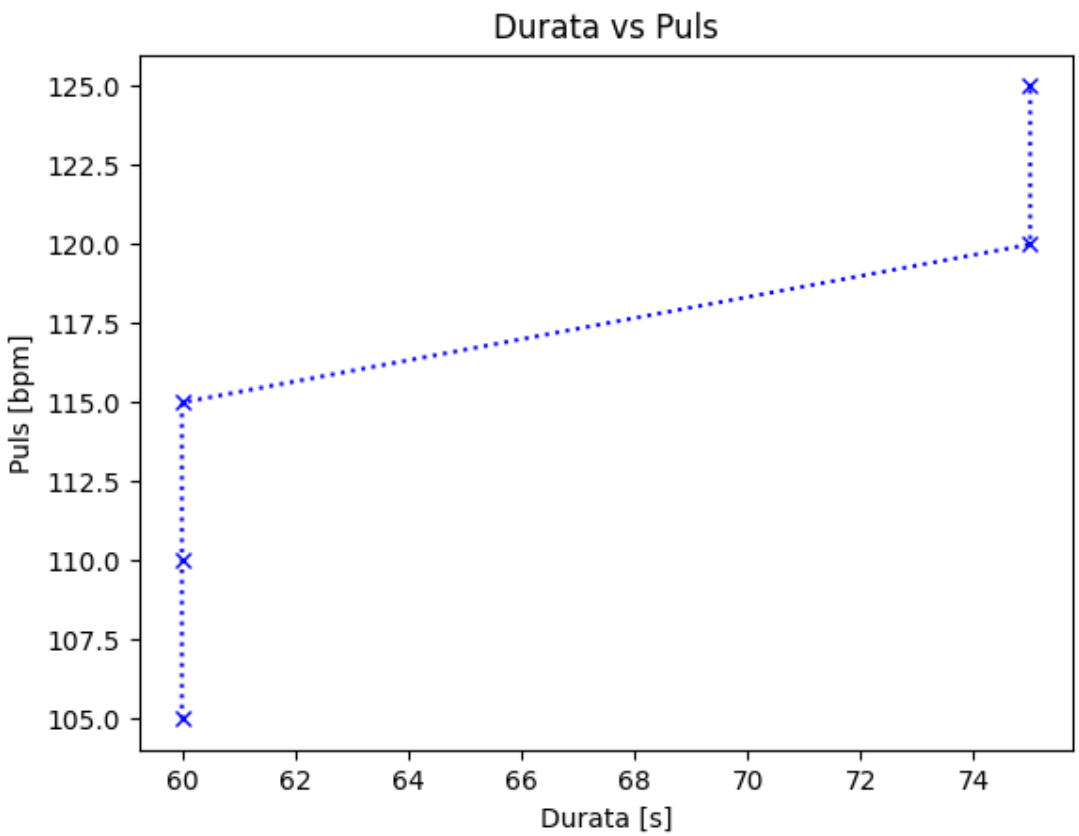
```
(.venv) PS C:\Users\grecu\Desktop\Tema 5> pip install -U matplotlib pandas
Collecting matplotlib
  Downloading matplotlib-3.10.9-cp313-cp313-win_amd64.whl.metadata (52 kB)
Requirement already satisfied: pandas in .\.venv\Lib\site-packages (3.0.2)
Collecting pandas
  Downloading pandas-3.0.3-cp313-cp313-win_amd64.whl.metadata (19 kB)
Collecting contourpy>=1.0.1 (from matplotlib)
  Downloading contourpy-1.3.3-cp313-cp313-win_amd64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib)
  Downloading fonttools-4.62.1-cp313-cp313-win_amd64.whl.metadata (119 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib)
  Downloading kiwisolver-1.5.0-cp313-cp313-win_amd64.whl.metadata (5.2 kB)
Requirement already satisfied: numpy>=1.23 in .\.venv\Lib\site-packages (from matplotlib) (2.4.4)
Requirement already satisfied: packaging>=20.0 in .\.venv\Lib\site-packages (from matplotlib) (26.2)
Collecting pillow>=8 (from matplotlib)
  Downloading pillow-12.2.0-cp313-cp313-win_amd64.whl.metadata (9.0 kB)
Collecting pyparsing>=3 (from matplotlib)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in .\.venv\Lib\site-packages (from matplotlib) (2.9.0.post0)
Requirement already satisfied: tzdata in .\.venv\Lib\site-packages (from pandas) (2026.2)
Requirement already satisfied: six>=1.5 in .\.venv\Lib\site-packages (from python-dateutil>=2.7->matplotlib) (1.17.0)
Downloading matplotlib-3.10.9-cp313-cp313-win_amd64.whl (8.2 MB)
   ---------------------------------------- 8.2/8.2 MB 26.1 MB/s  0:00:00
Downloading pandas-3.0.3-cp313-cp313-win_amd64.whl (9.8 MB)
   ---------------------------------------- 9.8/9.8 MB 66.4 MB/s  0:00:00
Downloading contourpy-1.3.3-cp313-cp313-win_amd64.whl (226 kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading fonttools-4.62.1-cp313-cp313-win_amd64.whl (2.3 MB)
   ---------------------------------------- 2.3/2.3 MB 27.5 MB/s  0:00:00
Downloading kiwisolver-1.5.0-cp313-cp313-win_amd64.whl (73 kB)
Downloading pillow-12.2.0-cp313-cp313-win_amd64.whl (7.1 MB)
   ---------------------------------------- 7.1/7.1 MB 47.1 MB/s  0:00:00
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Installing collected packages: pyparsing, pillow, kiwisolver, fonttools, cycler, contourpy, pandas, matplotlib
  Attempting uninstall: pandas
    Found existing installation: pandas 3.0.2
    Uninstalling pandas-3.0.2:
      Successfully uninstalled pandas-3.0.2
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.62.1 kiwisolver-1.5.0 matplotlib-3.10.9 pandas-3.0.3 pillow-12.2.0 pyparsing-3.3.2

[notice] A new release of pip is available: 26.0.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\grecu\Desktop\Tema 5>
```

În fesierul plotare_date.py am creat instrucțiunile pentru a respecta cerințele. Am importat la început pandas pentru a putea citi datele din csv și apoi pyplot din matplotlib pentru plotare. Am citit cu pandas conținutul fișierului ‘data.csv’, apoi am salvat în x_val primele 6 înregistrări folosind funcția head(6) pe data frame-ul creat anterior din citire. În y_val am salvat ultimele 6 înregistrări pentru coloanele ‘Durata’ si ‘Puls’. Am ales să le separ pe axa Ox si Oy pentru a le plota pentru ultima cerință mai frumos. Am creat 3 figuri pentru fiecare din cele 3 cerințe. În prima figură am plotat toate datele din fișier folosind funcția plot. Am adăugat un titlu și o legendă pentru a putea înțelege graficul. Pentru figura 2 am plotat fiecare coloană în parte cu diverși markeri, linii și culori pentru a putea face distincția între ele. Și aici am adăugat un titlu, o legendă și informații despre axa Ox și Oy (acele label-uri). În figura 3 am plotat pulsul în funcție de durată (similar cu figura 2). Fișierul se rulează din venv cu comanda python .\plotare_date.py. Output-ul se poate vizualiza în pozele atașate mai jos :

```
(.venv) PS C:\Users\grecu\Desktop\Tema 5> python .\plotare_date.py
   Durata  Puls  MaxPuls  Calorii
0      60   110      130    409.1
1      60   117      145    479.0
2      60   103      135    340.0
3      45   109      175    282.4
4      45   117      148    406.0
5      60   102      127    300.0
     Durata  Puls
164      60   105
165      60   110
166      60   115
167      75   120
168      75   125
```




Primele 6 valori pentru toate coloanele
500
400
300
200
100
Valoare
Durata
Puls
MaxPuls
Calorii
0
1
2
3
4
5
Inregistrare


Durata vs Puls
125.0
122.5
120.0
117.5
115.0
112.5
110.0
107.5
105.0
Puls [bpm]
60
62
64
66
68
70
72
74
Durata [s]

În continuare am decis să integrez vizualizarea graficelor într-o interfață web folosind Flask. Pentru a realiza acest lucru, trebuie să instalăm Flask în folder-ul venv cu comanda pip install flask:

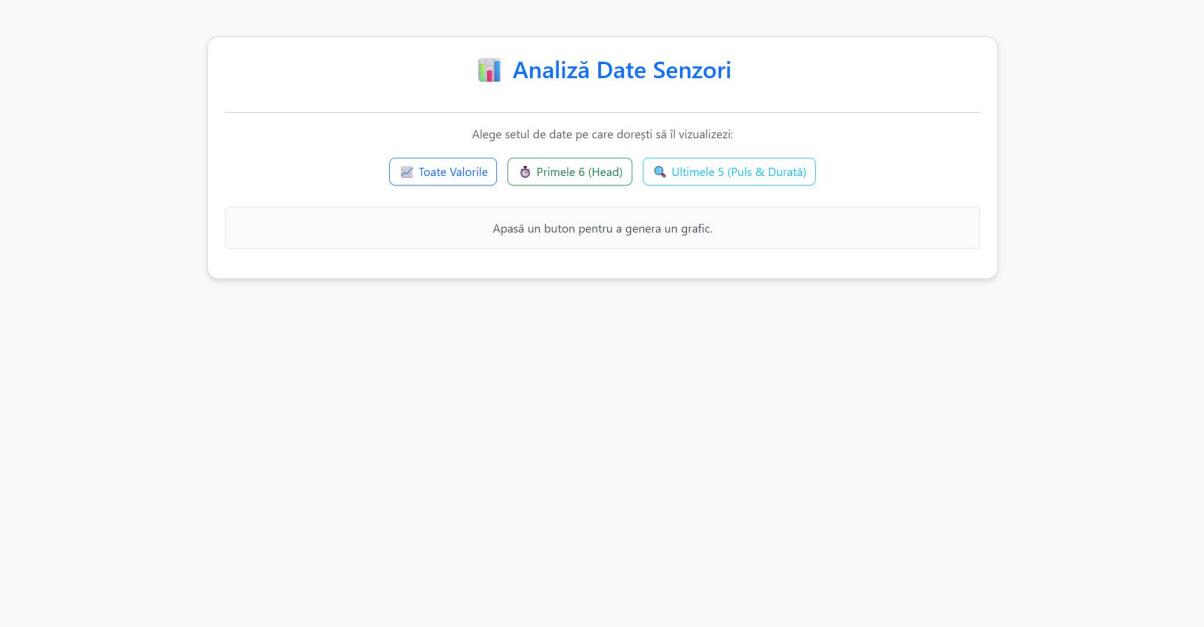
```
PS C:\Users\grecu\Desktop\Tema 5> .\.venv\Scripts\Activate.ps1
(.venv) PS C:\Users\grecu\Desktop\Tema 5> pip install flask
Collecting flask
  Downloading flask-3.1.3-py3-none-any.whl.metadata (3.2 kB)
Collecting blinker>=1.9.0 (from flask)
  Downloading blinker-1.9.0-py3-none-any.whl.metadata (1.6 kB)
Collecting click>=8.1.3 (from flask)
  Downloading click-8.3.3-py3-none-any.whl.metadata (2.6 kB)
Collecting itsdangerous>=2.2.0 (from flask)
  Downloading itsdangerous-2.2.0-py3-none-any.whl.metadata (1.9 kB)
Collecting jinja2>=3.1.2 (from flask)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting markupsafe>=2.1.1 (from flask)
  Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl.metadata (2.8 kB)
Collecting werkzeug>=3.1.0 (from flask)
  Downloading werkzeug-3.1.8-py3-none-any.whl.metadata (4.0 kB)
Requirement already satisfied: colorama in .\.venv\Lib\site-packages (from click>=8.1.3->flask) (0.4.6)
Downloading flask-3.1.3-py3-none-any.whl (103 kB)
Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
Downloading click-8.3.3-py3-none-any.whl (110 kB)
Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
Using cached jinja2-3.1.6-py3-none-any.whl (134 kB)
Using cached markupsafe-3.0.3-cp313-cp313-win_amd64.whl (15 kB)
Downloading werkzeug-3.1.8-py3-none-any.whl (226 kB)
Installing collected packages: markupsafe, itsdangerous, click, blinker, werkzeug, jinja2, flask
Successfully installed blinker-1.9.0 click-8.3.3 flask-3.1.3 itsdangerous-2.2.0 jinja2-3.1.6 markupsafe-3.0.3 werkzeug-3.1.8

[notice] A new release of pip is available: 26.0.1 -> 26.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Users\grecu\Desktop\Tema 5>
```

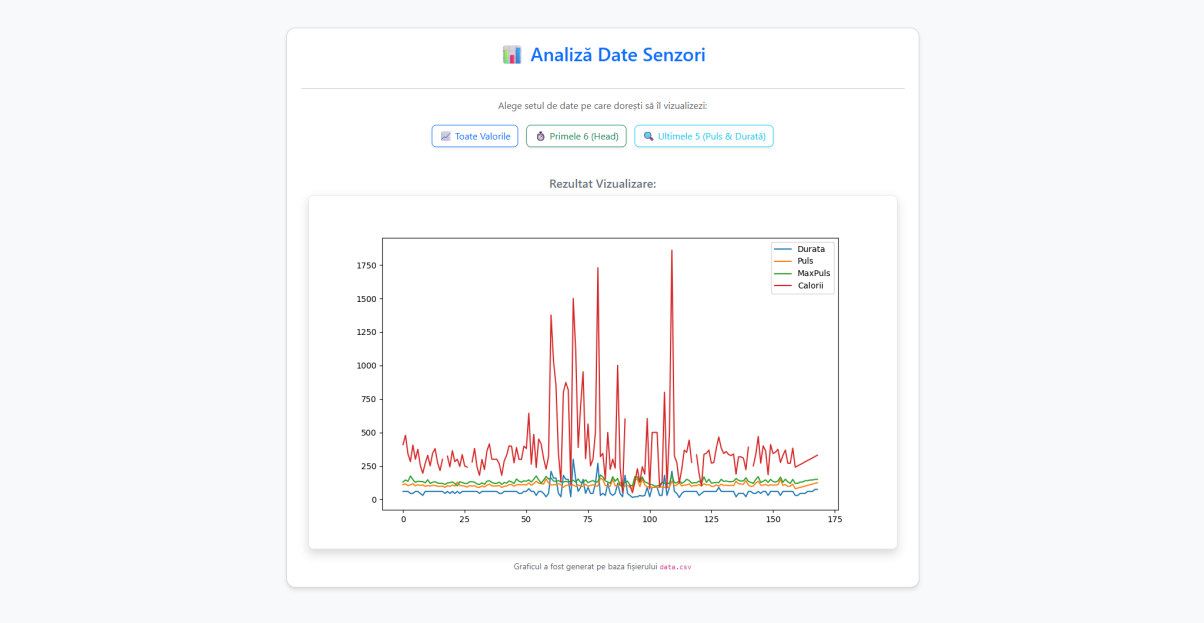
Structura fișierelor pentru Flask este următoarea :

- În folder-ul template se pun paginile HTML
- În folder-ul static se pun lucruri statice (pagini, cod CSS, imagini etc.)
- În folder-ul în care se află proiectul vom avea aplicația noastră app.py și fișierul data.csv. Pentru a rula aplicația folosim python app.py

În fișierul app.py am importat Flask, render_templat, request, pandas, matplotlib și os. De asemenea este nevoie să specificăm ca matplotlib să utilizeze ‘Agg’ întrucât server-ul nu are un monitor (reprezintă modul non-interactiv -> graficul generat este salvat în memoria RAM). Dacă rulăm scriptul inițial se deschidea automat o fereastră în care apăreau graficele, dar aici suntem pe un server web și nu putem face acest lucru, de aceea folosim ‘Agg’. Am definit 2 rute, cea default (root – cu /) și una pentru grafice (plot). Ruta root afișează pagina HTML denumita index.html în care am niște butoane pentru cele 3 cerințe ca să fie afișate graficele. Acele butoane se afla într-un formular și sunt trimise prin metoda POST. Pentru ruta de plot, am citit fișierul data.csv și am salvat într-o variabilă tipul graficului pe care îl iau prin metoda POST de la formularul din pagina index.html (acel name). El va prelua de la butoane valoarea de pe câmpul value. Am definit niște dimensiuni pentru graficele care vor fi afișate. Într-un if verific pentru fiecare tip de buton apăsat și afișez ce aveam și în fișierul inițial (plotare_date.py). Acum ne folosim de modulul os. Output-ul pe care îl generează matplotlib este salvat momentan în memoria RAM. Folosim savefile din

matplotlib pentru a-l salva sub formă de fișier .png cu numele grafic.png. Acel filepath ne spune ca el să caute în folder-ul static un fișier numit grafic.png (cel creat anterior). La final ne folosim de close pentru a fi șters din memoria RAM. La final afișăm în pagina index.html imaginea grafic.png aferentă butonului pe care am apăsat. Imaginea va fi practic suprascrisă de fiecare dată când apăsam pe buton, el nu salvează imaginile și le refolosește, de fiecare dată reface graficul, îl transformă în .png și îl afișează. Am activat și opțiunea de debug pentru a vedea dacă avem diverse mesaje de eroare sau probleme pentru a le putea rezolva.





Sitografie :

- **Testarea unitară cu Pytest (Ghid VS Code):** https://code.visualstudio.com/docs/python/testing
- **Ghid utilizare Pytest (TutorialsPoint):** https://www.tutorialspoint.com/pytest/index.htm
- **Pandas - Analiza datelor din CSV:** https://www.w3schools.com/python/pandas/default.asp
- **Matplotlib - Crearea graficelor:** https://www.w3schools.com/python/matplotlib_intro.asp
- **Matplotlib Backend 'Agg' (Documentație oficială):** https://matplotlib.org/stable/users/explain/backends.html
- **Documentație Flask:** https://www.geeksforgeeks.org/python/flask-tutorial/
- **Python os module - Manipularea căilor de fișiere:** https://docs.python.org/3/library/os.path.html
- **Flask Static Files:** https://flask.palletsprojects.com/en/stable/quickstart/#static-files